

CSE 344 — System Programming

Homework 5 Report: Priority-Based Cargo Delivery Simulation (`cargoGTU`)

Salih Cengiz

Student ID: 220104004007

Spring 2026

Contents

1. System Design	2
1.1 Thread pool and roles	2
1.2 Courier lifecycle	2
1.3 Module layout	2
1.4 Control flow diagram	3
2. Priority Queue Design	4
2.1 Ordering rule	4
2.2 Data structure	4
2.3 Why a heap	4
3. Synchronization Strategy	5
4. SIGINT Handling	6
5. Test Scenarios	7
5.1 Scenario 1 — <code>N=3, 10orders_mix.txt</code>	7
5.2 Scenario 2 — <code>N=10, 10orders_economy.txt</code>	9
5.3 Scenario 3 — <code>N=2, 20orders_mix.txt</code> , SIGINT after ~2 seconds	11
5.4 Scenario 4 — Valgrind memory check	13
5.5 Scenario 5 — <code>N=10, 20orders_mix.txt</code> , repeated runs	15
6. Challenges & Solutions	17

CSE 344 — System Programming

Homework #5

Priority-Based Cargo Delivery Simulation

This report documents the design and implementation of `cargoGTU`, a POSIX thread-pool simulator that schedules delivery orders from a priority queue, logs a structured trace to `stdout`, writes a summary to a **statistics file** (`-s`), and performs a **safe SIGINT shutdown**. The implementation targets **Debian 12 (64-bit)** and is built with `gcc -Wall -Wextra -std=c11 -pthread`.

1. System Design

1.1 Thread pool and roles

The program uses exactly **N** courier threads created at startup (**-n**), plus one **dedicated signal-handling thread** that waits on **SIGINT** with **sigwait()** (no asynchronous handler that runs unsafe code). **No per-order threads** are created during the shift.

Role	Responsibility
Main thread	Parses CLI, blocks SIGINT in the process-wide mask before creating threads, builds the priority queue, logs ORDER_QUEUED in input order, starts the signal thread and all couriers, pthread_joins couriers, notifies the signal thread to exit on natural completion, prints SHIFT_END / SHUTDOWN_COMPLETE , writes the stats file, and releases resources.
Courier threads	Loop: under queue_mutex , wait on the condition variable if the queue is empty and shutdown is not requested; otherwise pop the minimum (highest scheduling priority) order, simulate delivery with nanosleep , update atomic global counters and per-courier summary, log DELIVERY_COMPLETE , repeat until shutdown with an empty queue; then log SHIFT_OVER .
Signal thread	Blocks on sigwait(SIGINT) . On a real interrupt: sets global shutdown, locks the queue, logs SIGINT_RECEIVED , drains pending orders with ORDER_CANCELLED and increments the cancelled atomic counter, broadcasts the condition variable, unlocks. If woken artificially after normal completion (g_natural_shutdown), exits without printing SIGINT artefacts.

1.2 Courier lifecycle

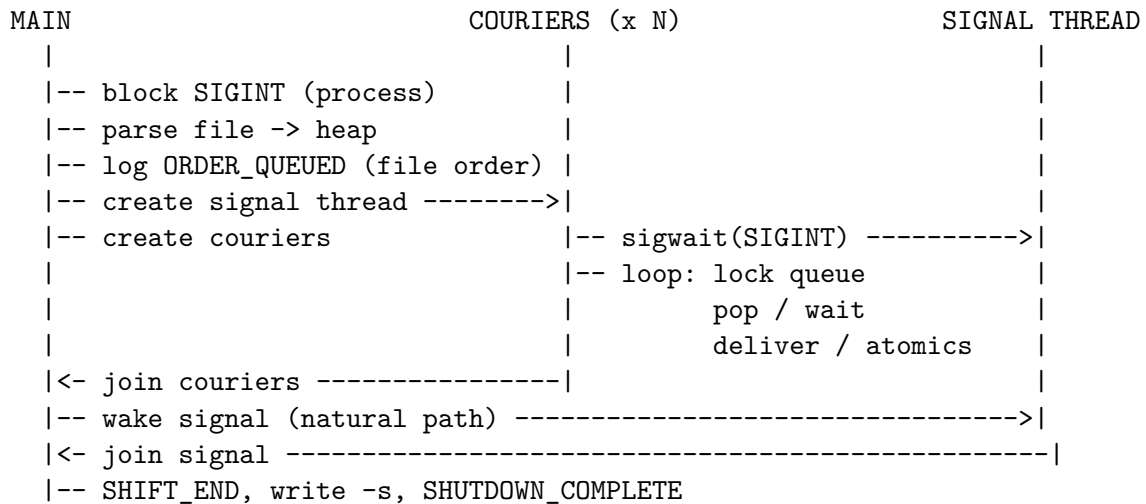
Courier behavior follows the homework requirement: **pthread_cond_wait** when the queue is empty; **busy-wait loops are not used**.

1. Courier acquires **g_queue_mutex**.
2. If **all work is accounted for** (**completed + cancelled >= total orders**), set **g_shutdown** and **pthread_cond_broadcast** so every courier eventually observes shutdown (natural completion path).
3. If the queue is **empty** and shutdown is **not** yet active, log **WAITING** and **pthread_cond_wait** until signaled.
4. If shutdown is active **and** the queue stays empty, **exit loop** → **SHIFT_OVER**.
5. Otherwise **pop** the next order (**pq_pop_min**), still under the mutex → log **DELIVERY_START** → **unlock**.
6. **Simulate delivery** (1 simulation unit = **500 ms**).
7. **atomic_fetch_add** on **g_completed_orders** and **g_total_delivery_time_ms**; update this courier's **g_courier_stats[i]**.
8. Log **DELIVERY_COMPLETE** and loop.

1.3 Module layout

File	Purpose
main.c	CLI, initialization, load orders, enqueue, thread creation/join, final output and cleanup.
priority_queue.c / priority_queue.h	Binary min-heap implementation of the shared priority queue.
parser.c / parser.h	Robust line-by-line parsing; invalid lines skipped silently.
courier.c / courier.h	Courier thread entry point.
signal_thread.c / signal_thread.h	sigwait-based SIGINT handling.
stats.c / stats.h	Writes SHIFT_SUMMARY and COURIER_STATS to -s.
log.c	All printf logging guarded by g_log_mutex .
cargo.h	Shared types, globals, and log prototypes.

1.4 Control flow diagram



Design rationale: separating **delivery work** (outside the queue mutex) from **queue mutations** minimizes contention. A dedicated **sigwait** thread avoids **async-signal-unsafe** calls inside a classical signal handler while still honoring “pending cancelled, broadcast waiters.”

2. Priority Queue Design

2.1 Ordering rule

The scheduling order is **EXPRESS (level 1) > STANDARD (2) > ECONOMY (3)**. When priorities tie, **id** breaks the tie (**smaller id first**). Internally priorities are encoded as numeric enums `PRI0_EXPRESS = 1`, etc., so smaller numeric priority wins.

2.2 Data structure

Orders live in a **binary min-heap** (`pqueue_t`: dynamic array `data[]`, `size`, `cap`). The heap property uses a comparator `order_less(a,b)` that is true when **a should be served before b**:

- `a.priority < b.priority`, or
- `a.priority == b.priority && a.id < b.id`

Push appends at the end and **sifts up**; **pop_min** takes the root, moves the last element to the root, and **sifts down**. Both are $O(\log n)$.

2.3 Why a heap

A heap gives **logarithmic** enqueue/dequeue with **simple array storage** and clear invariants for “always take the global minimum under the comparator,” which is exactly “highest business priority, then lowest id.” The shared heap is only accessed while `g_queue_mutex` is held.

3. Synchronization Strategy

Resource	Protection	Notes
Priority queue (<code>g_queue</code>)	<code>pthread_mutex_t g_queue_mutex</code> + <code>pthread_cond_t g_queue_cond</code>	All <code>pq_*</code> mutations and <code>pthread_cond_wait</code> / <code>broadcast</code> occur with this mutex.
Stdout log lines	<code>pthread_mutex_t g_log_mutex</code>	Held around every <code>printf</code> (and <code>fflush</code>) so multi- <code>printf</code> lines cannot interleave between threads.
<code>completed_orders</code> , <code>cancelled_orders</code> , <code>total_delivery_time_ms</code>	C11 <code>_Atomic</code> (<code>stdatomic.h</code>) with <code>atomic_fetch_add</code> / <code>atomic_load</code>	Mutex-free counter updates per specification; avoids the grading penalty for mutex-based global stats.
Shutdown coordination	<code>_Atomic int g_shutdown</code> , <code>_Atomic</code> <code>int g_natural_shutdown</code>	Couriers and the signal thread agree on termination; <code>pthread_cond_broadcast</code> wakes waiters after cancellation or completion.

Lock-ordering discipline: `log_*` functions never acquire `g_queue_mutex`, so holding `g_queue_mutex` while logging (e.g. `SIGINT_RECEIVED`, `ORDER_CANCELLED`) is safe. Signal thread logs cancellation while holding `g_queue_mutex` to keep `pending_orders` consistent with the actual drain.

4. SIGINT Handling

1. Before any threads are spawned, **main** calls **pthread_sigmask(SIG_BLOCK, {SIGINT}, NULL)** so delivery threads do not observe SIGINT asynchronously.
2. A **signal thread** waits on **sigwait(&set)** where **set = {SIGINT}**.
3. On **SIGINT**:
 - If **g_natural_shutdown** is set, exit **silently** (spurious wakeup path after normal shutdown).
 - Else set **g_shutdown**, lock **g_queue_mutex**, compute **pending_orders = pq_size()**, print **SIGINT_RECEIVED**, repeatedly **pq_pop_min**, each time **atomic_fetch_add(&g_cancelled_orders, 1)** and **ORDER_CANCELLED**, then **pthread_cond_broadcast**, unlock.
4. **Couriers in cond_wait** wake up: with shutdown and empty queue, they **do not start new work** and exit with **SHIFT_OVER**.
5. **Couriers mid-delivery** finish **nanosleep** (delivery is **never** aborted mid-flight), complete **DELIVERY_COMPLETE**, then on the next iteration observe shutdown and exit.
6. **Main pthread_joins** all couriers, sets **g_natural_shutdown**, sends **pthread_kill(signal_tid, SIGINT)** to unblock **sigwait**, **pthread_joins** the signal thread, prints **SHIFT_END**, writes **-s**, prints **SHUTDOWN_COMPLETE**.

5. Test Scenarios

Base command pattern:

```
./cargoGTU -n <N> -i <input_file> -s stats.txt
```

5.1 Scenario 1 — N=3, 10orders_mix.txt

- **Command:** `./cargoGTU -n 3 -i HW5_test_files/tests/10orders_mix.txt -s stats.txt`
- **Verification:** All 10 orders completed. Stats file written correctly (completed=10 cancelled=0 total_time=58000ms).
- **Screenshot:**

```
Scenario 1 - n=3, 10orders_mix.txt

$ "/mnt/c/Users/Cengiz/Desktop/Spring 2026/CSE344-Systems_Programming/Hws/HW5/cargoGTU" -n 3 -i
"/mnt/c/Users/Cengiz/Desktop/Spring
2026/CSE344-Systems_Programming/Hws/HW5/HW5_test_files/tests/10orders_mix.txt" -s
"/mnt/c/Users/Cengiz/Desktop/Spring 2026/CSE344-Systems_Programming/Hws/HW5/captures/_stats_scenario1.txt"
[CARGOGTU] SHIFT_START couriers=3 orders=10
[CARGOGTU] ORDER_QUEUED id=1 recipient=Ahmet_Yilmaz priority=EXPRESS duration=8
[CARGOGTU] ORDER_QUEUED id=2 recipient=Ayse_Kaya priority=STANDARD duration=12
[CARGOGTU] ORDER_QUEUED id=3 recipient=Mehmet_Demir priority=ECONOMY duration=20
[CARGOGTU] ORDER_QUEUED id=4 recipient=Fatma_Sahin priority=EXPRESS duration=6
[CARGOGTU] ORDER_QUEUED id=5 recipient=Ali_Celik priority=STANDARD duration=9
[CARGOGTU] ORDER_QUEUED id=6 recipient=Zeynep_Arslan priority=ECONOMY duration=15
[CARGOGTU] ORDER_QUEUED id=7 recipient=Mustafa_Koc priority=EXPRESS duration=7
[CARGOGTU] ORDER_QUEUED id=8 recipient=Elif_Demir priority=STANDARD duration=11
[CARGOGTU] ORDER_QUEUED id=9 recipient=Hasan_Ozturk priority=ECONOMY duration=18
[CARGOGTU] ORDER_QUEUED id=10 recipient=Merve_Yildiz priority=STANDARD duration=10
[COURIER-1] DELIVERY_START id=1 recipient=Ahmet_Yilmaz priority=EXPRESS
[COURIER-2] DELIVERY_START id=4 recipient=Fatma_Sahin priority=EXPRESS
[COURIER-3] DELIVERY_START id=7 recipient=Mustafa_Koc priority=EXPRESS
[COURIER-2] DELIVERY_COMPLETE id=4 recipient=Fatma_Sahin duration=3000ms
[COURIER-2] DELIVERY_START id=2 recipient=Ayse_Kaya priority=STANDARD
[COURIER-3] DELIVERY_COMPLETE id=7 recipient=Mustafa_Koc duration=3500ms
[COURIER-3] DELIVERY_START id=5 recipient=Ali_Celik priority=STANDARD
[COURIER-1] DELIVERY_COMPLETE id=1 recipient=Ahmet_Yilmaz duration=4000ms
[COURIER-1] DELIVERY_START id=8 recipient=Elif_Demir priority=STANDARD
[COURIER-3] DELIVERY_COMPLETE id=5 recipient=Ali_Celik duration=4500ms
[COURIER-3] DELIVERY_START id=10 recipient=Merve_Yildiz priority=STANDARD
[COURIER-2] DELIVERY_COMPLETE id=2 recipient=Ayse_Kaya duration=6000ms
[COURIER-2] DELIVERY_START id=3 recipient=Mehmet_Demir priority=ECONOMY
[COURIER-1] DELIVERY_COMPLETE id=8 recipient=Elif_Demir duration=5500ms
[COURIER-1] DELIVERY_START id=6 recipient=Zeynep_Arslan priority=ECONOMY
[COURIER-3] DELIVERY_COMPLETE id=10 recipient=Merve_Yildiz duration=5000ms
[COURIER-3] DELIVERY_START id=9 recipient=Hasan_Ozturk priority=ECONOMY
[COURIER-1] DELIVERY_COMPLETE id=6 recipient=Zeynep_Arslan duration=7500ms
[COURIER-1] WAITING
[COURIER-2] DELIVERY_COMPLETE id=3 recipient=Mehmet_Demir duration=10000ms
[COURIER-2] WAITING
[COURIER-3] DELIVERY_COMPLETE id=9 recipient=Hasan_Ozturk duration=9000ms
[COURIER-3] SHIFT_OVER
[COURIER-1] SHIFT_OVER
[COURIER-2] SHIFT_OVER
[CARGOGTU] SHIFT_END completed=10 cancelled=0 total_time=58000ms
[CARGOGTU] SHUTDOWN_COMPLETE

$ cat captures/_stats_scenario1.txt
SHIFT_SUMMARY
Total orders : 10
Completed : 10
Cancelled : 0
Total time : 58000ms
Avg per order : 5800ms

COURIER_STATS
Courier-1 completed=3 total_time=17000ms
Courier-2 completed=3 total_time=19000ms
Courier-3 completed=4 total_time=22000ms
```

Figure 1: Scenario 1 — full console from prompt to exit and stats verification

5.2 Scenario 2 — N=10, 10orders_economy.txt

- **Command:** `./cargoGTU -n 10 -i HW5_test_files/tests/10orders_economy.txt -s stats.txt`
- **Verification:** All 10 orders completed. Since all orders share the same priority (ECONOMY), lower id orders are taken first (id=1,2,...,10 in order). Multiple couriers participate simultaneously.
- **Screenshot:**

```
Scenario 2 - n=10, economy

$ "/mnt/c/Users/Cengiz/Desktop/Spring 2026/CSE344-Systems_Programming/HWs/HW5/cargoGTU" -n 10 -i
"/mnt/c/Users/Cengiz/Desktop/Spring
2026/CSE344-Systems_Programming/HWs/HW5/HW5_test_files/tests/10orders_economy.txt" -s
"/mnt/c/Users/Cengiz/Desktop/Spring 2026/CSE344-Systems_Programming/HWs/HW5/captures/_stats_scenario2.txt"
[CARGOGTU] SHIFT_START couriers=10 orders=10
[CARGOGTU] ORDER_QUEUED id=1 recipient=Ahmet_Yilmaz priority=ECONOMY duration=8
[CARGOGTU] ORDER_QUEUED id=2 recipient=Ayşe_Kaya priority=ECONOMY duration=10
[CARGOGTU] ORDER_QUEUED id=3 recipient=Mehmet_Demir priority=ECONOMY duration=7
[CARGOGTU] ORDER_QUEUED id=4 recipient=Fatma_Sahin priority=ECONOMY duration=12
[CARGOGTU] ORDER_QUEUED id=5 recipient=Ali_Celik priority=ECONOMY duration=9
[CARGOGTU] ORDER_QUEUED id=6 recipient=Zeynep_Arslan priority=ECONOMY duration=11
[CARGOGTU] ORDER_QUEUED id=7 recipient=Mustafa_Koc priority=ECONOMY duration=8
[CARGOGTU] ORDER_QUEUED id=8 recipient=Elif_Demir priority=ECONOMY duration=6
[CARGOGTU] ORDER_QUEUED id=9 recipient=Hasan_Ozturk priority=ECONOMY duration=10
[CARGOGTU] ORDER_QUEUED id=10 recipient=Merve_Yildiz priority=ECONOMY duration=9
[COURIER-1] DELIVERY_START id=1 recipient=Ahmet_Yilmaz priority=ECONOMY
[COURIER-2] DELIVERY_START id=2 recipient=Ayşe_Kaya priority=ECONOMY
[COURIER-3] DELIVERY_START id=3 recipient=Mehmet_Demir priority=ECONOMY
[COURIER-4] DELIVERY_START id=4 recipient=Fatma_Sahin priority=ECONOMY
[COURIER-5] DELIVERY_START id=5 recipient=Ali_Celik priority=ECONOMY
[COURIER-6] DELIVERY_START id=6 recipient=Zeynep_Arslan priority=ECONOMY
[COURIER-7] DELIVERY_START id=7 recipient=Mustafa_Koc priority=ECONOMY
[COURIER-8] DELIVERY_START id=8 recipient=Elif_Demir priority=ECONOMY
[COURIER-9] DELIVERY_START id=9 recipient=Hasan_Ozturk priority=ECONOMY
[COURIER-10] DELIVERY_START id=10 recipient=Merve_Yildiz priority=ECONOMY
[COURIER-8] DELIVERY_COMPLETE id=8 recipient=Elif_Demir duration=3000ms
[COURIER-8] WAITING
[COURIER-3] DELIVERY_COMPLETE id=3 recipient=Mehmet_Demir duration=3500ms
[COURIER-3] WAITING
[COURIER-1] DELIVERY_COMPLETE id=1 recipient=Ahmet_Yilmaz duration=4000ms
[COURIER-1] WAITING
[COURIER-7] DELIVERY_COMPLETE id=7 recipient=Mustafa_Koc duration=4000ms
[COURIER-7] WAITING
[COURIER-5] DELIVERY_COMPLETE id=5 recipient=Ali_Celik duration=4500ms
[COURIER-5] WAITING
[COURIER-10] DELIVERY_COMPLETE id=10 recipient=Merve_Yildiz duration=4500ms
[COURIER-10] WAITING
[COURIER-2] DELIVERY_COMPLETE id=2 recipient=Ayşe_Kaya duration=5000ms
[COURIER-2] WAITING
[COURIER-9] DELIVERY_COMPLETE id=9 recipient=Hasan_Ozturk duration=5000ms
[COURIER-9] WAITING
[COURIER-6] DELIVERY_COMPLETE id=6 recipient=Zeynep_Arslan duration=5500ms
[COURIER-6] WAITING
[COURIER-4] DELIVERY_COMPLETE id=4 recipient=Fatma_Sahin duration=6000ms
[COURIER-4] SHIFT_OVER
[COURIER-9] SHIFT_OVER
[COURIER-2] SHIFT_OVER
[COURIER-6] SHIFT_OVER
[COURIER-5] SHIFT_OVER
[COURIER-8] SHIFT_OVER
[COURIER-7] SHIFT_OVER
[COURIER-10] SHIFT_OVER
[COURIER-3] SHIFT_OVER
[COURIER-1] SHIFT_OVER
[CARGOGTU] SHIFT_END completed=10 cancelled=0 total_time=45000ms
[CARGOGTU] SHUTDOWN_COMPLETE
```

Figure 2: Scenario 2 — all economy, ten couriers, id-ordered dispatch

5.3 Scenario 3 — N=2, 20orders_mix.txt, SIGINT after ~2 seconds

- **Command:** `./cargoGTU -n 2 -i HW5_test_files/tests/20orders_mix.txt -s stats.txt` then `kill -INT <pid>` after ~2 s.
- **Verification:** SIGINT_RECEIVED appears; active deliveries complete before SHIFT_OVER; all pending orders show ORDER_CANCELLED; SHIFT_END shows `completed + cancelled = 20`. ps confirms no stranded process after exit.
- **Screenshot:**

```
Scenario 3 — SIGINT (~2s), n=2

$ "/mnt/c/Users/Cengiz/Desktop/Spring 2026/CSE344-Systems_Programming/Hws/HW5/cargoGTU" -n 2 -i
"/mnt/c/Users/Cengiz/Desktop/Spring
2026/CSE344-Systems_Programming/Hws/HW5/HW5_test_files/tests/20orders_mix.txt" -s
"/mnt/c/Users/Cengiz/Desktop/Spring 2026/CSE344-Systems_Programming/Hws/HW5/captures/_stats_sigint.txt" #
SIGINT after ~2s

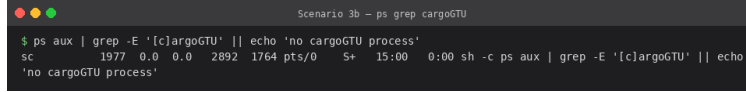
[CARGOGTU] SHIFT_START couriers=2 orders=20
[CARGOGTU] ORDER_QUEUED id=1 recipient=Ahmet_Yilmaz priority=EXPRESS duration=1
[CARGOGTU] ORDER_QUEUED id=2 recipient=Ayşe_Kaya priority=ECONOMY duration=3
[CARGOGTU] ORDER_QUEUED id=3 recipient=Mehmet_Demir priority=STANDARD duration=2
[CARGOGTU] ORDER_QUEUED id=4 recipient=Fatma_Sahin priority=EXPRESS duration=1
[CARGOGTU] ORDER_QUEUED id=5 recipient=Ali_Celik priority=STANDARD duration=3
[CARGOGTU] ORDER_QUEUED id=6 recipient=Zeynep_Arslan priority=ECONOMY duration=2
[CARGOGTU] ORDER_QUEUED id=7 recipient=Mustafa_Koc priority=EXPRESS duration=1
[CARGOGTU] ORDER_QUEUED id=8 recipient=Elif_Demir priority=STANDARD duration=2
[CARGOGTU] ORDER_QUEUED id=9 recipient=Hasan_Ozturk priority=ECONOMY duration=3
[CARGOGTU] ORDER_QUEUED id=10 recipient=Merve_Yildiz priority=EXPRESS duration=1
[CARGOGTU] ORDER_QUEUED id=11 recipient=Emre_Celik priority=STANDARD duration=2
[CARGOGTU] ORDER_QUEUED id=12 recipient=Selin_Kaya priority=ECONOMY duration=3
[CARGOGTU] ORDER_QUEUED id=13 recipient=Burak_Arslan priority=EXPRESS duration=1
[CARGOGTU] ORDER_QUEUED id=14 recipient=Dilan_Ozkan priority=STANDARD duration=2
[CARGOGTU] ORDER_QUEUED id=15 recipient=Kemal_Sahin priority=ECONOMY duration=3
[CARGOGTU] ORDER_QUEUED id=16 recipient=Rana_Yilmaz priority=EXPRESS duration=2
[CARGOGTU] ORDER_QUEUED id=17 recipient=Tarik_Demir priority=STANDARD duration=1
[CARGOGTU] ORDER_QUEUED id=18 recipient=Pinar_Koc priority=ECONOMY duration=2
[CARGOGTU] ORDER_QUEUED id=19 recipient=Oguz_Yildiz priority=EXPRESS duration=3
[CARGOGTU] ORDER_QUEUED id=20 recipient=Ceren_Arslan priority=STANDARD duration=1
[COURIER-1] DELIVERY_START id=1 recipient=Ahmet_Yilmaz priority=EXPRESS
[COURIER-2] DELIVERY_START id=4 recipient=Fatma_Sahin priority=EXPRESS
[COURIER-1] DELIVERY_COMPLETE id=1 recipient=Ahmet_Yilmaz duration=500ms
[COURIER-1] DELIVERY_START id=7 recipient=Mustafa_Koc priority=EXPRESS
[COURIER-2] DELIVERY_COMPLETE id=4 recipient=Fatma_Sahin duration=500ms
[COURIER-2] DELIVERY_START id=10 recipient=Merve_Yildiz priority=EXPRESS
[COURIER-1] DELIVERY_COMPLETE id=7 recipient=Mustafa_Koc duration=500ms
[COURIER-1] DELIVERY_START id=13 recipient=Burak_Arslan priority=EXPRESS
[COURIER-2] DELIVERY_COMPLETE id=10 recipient=Merve_Yildiz duration=500ms
[COURIER-2] DELIVERY_START id=16 recipient=Rana_Yilmaz priority=EXPRESS
[COURIER-1] DELIVERY_COMPLETE id=13 recipient=Burak_Arslan duration=500ms
[COURIER-1] DELIVERY_START id=19 recipient=Oguz_Yildiz priority=EXPRESS
[CARGOGTU] SIGINT_RECEIVED pending_orders=13
[CARGOGTU] ORDER_CANCELLED id=3 recipient=Mehmet_Demir priority=STANDARD
[CARGOGTU] ORDER_CANCELLED id=5 recipient=Ali_Celik priority=STANDARD
[CARGOGTU] ORDER_CANCELLED id=8 recipient=Elif_Demir priority=STANDARD
[CARGOGTU] ORDER_CANCELLED id=11 recipient=Emre_Celik priority=STANDARD
[CARGOGTU] ORDER_CANCELLED id=14 recipient=Dilan_Ozkan priority=STANDARD
[CARGOGTU] ORDER_CANCELLED id=17 recipient=Tarik_Demir priority=STANDARD
[CARGOGTU] ORDER_CANCELLED id=20 recipient=Ceren_Arslan priority=STANDARD
[CARGOGTU] ORDER_CANCELLED id=2 recipient=Ayşe_Kaya priority=ECONOMY
[CARGOGTU] ORDER_CANCELLED id=6 recipient=Zeynep_Arslan priority=ECONOMY
[CARGOGTU] ORDER_CANCELLED id=9 recipient=Hasan_Ozturk priority=ECONOMY
[CARGOGTU] ORDER_CANCELLED id=12 recipient=Selin_Kaya priority=ECONOMY
[CARGOGTU] ORDER_CANCELLED id=15 recipient=Kemal_Sahin priority=ECONOMY
[CARGOGTU] ORDER_CANCELLED id=18 recipient=Pinar_Koc priority=ECONOMY
[COURIER-2] DELIVERY_COMPLETE id=16 recipient=Rana_Yilmaz duration=1000ms
[COURIER-2] SHIFT_OVER
[COURIER-1] DELIVERY_COMPLETE id=19 recipient=Oguz_Yildiz duration=1500ms
[COURIER-1] SHIFT_OVER
[CARGOGTU] SHIFT_END completed=7 cancelled=13 total_time=5000ms
[CARGOGTU] SHUTDOWN_COMPLETE

$ cat captures/_stats_sigint.txt
SHIFT_SUMMARY
Total orders : 20
Completed : 7
Cancelled : 13
Total time : 5000ms
Avg per order : 714ms

COURIER_STATS
Courier-1 completed=4 total_time=3000ms
Courier-2 completed=3 total_time=2000ms
```

Figure 3: Scenario 3 — SIGINT shutdown, active deliveries finish, pending cancelled

- Screenshot (ps check after exit):



```
Scenario 3b - ps grep cargoGTU
$ ps aux | grep -E '[c]argoGTU' || echo 'no cargoGTU process'
sc 1977 0.0 0.0 2892 1764 pts/0 S+ 15:00 0:00 sh -c ps aux | grep -E '[c]argoGTU' || echo 'no cargoGTU process'
```

Figure 4: Scenario 3b — ps confirms no cargoGTU process remains

5.4 Scenario 4 — Valgrind memory check

- **Command:** `valgrind --leak-check=full --show-leak-kinds=all ./cargoGTU -n 3 -i HW5_test_files/tests/10orders_mix.txt -s stats.txt`
- **Verification:** Valgrind reports ERROR SUMMARY: 0 errors from 0 contexts and All heap blocks were freed -- no leaks are possible.
- **Screenshot:**

```
Scenario 4 - valgrind

$ /usr/bin/valgrind --leak-check=full --show-leak-kinds=all "/mnt/c/Users/Cengiz/Desktop/Spring
2026/CSE344-Systems_Programming/Hws/HW5/cargoGTU" -n 3 -i "/mnt/c/Users/Cengiz/Desktop/Spring
2026/CSE344-Systems_Programming/Hws/HW5/HW5_test_files/tests/10orders_mix.txt" -s
"/mnt/c/Users/Cengiz/Desktop/Spring 2026/CSE344-Systems_Programming/Hws/HW5/captures/_stats_valgrind.txt"

==1881== Memcheck, a memory error detector
==1881== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==1881== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==1881== Command: /mnt/c/Users/Cengiz/Desktop/Spring\ 2026/CSE344-Systems_Programming/Hws/HW5/cargoGTU -n 3 -i
/mnt/c/Users/Cengiz/Desktop/Spring\
2026/CSE344-Systems_Programming/Hws/HW5/HW5_test_files/tests/10orders_mix.txt -s
/mnt/c/Users/Cengiz/Desktop/Spring\ 2026/CSE344-Systems_Programming/Hws/HW5/captures/_stats_valgrind.txt
==1881==
[CARGOGTU] SHIFT_START couriers=3 orders=10
[CARGOGTU] ORDER_QUEUE id=1 recipient=Ahmet_Yilmaz priority=EXPRESS duration=8
[CARGOGTU] ORDER_QUEUE id=2 recipient=Ayse_Kaya priority=STANDARD duration=12
[CARGOGTU] ORDER_QUEUE id=3 recipient=Mehmet_Demir priority=ECONOMY duration=20
[CARGOGTU] ORDER_QUEUE id=4 recipient=Fatma_Sahin priority=EXPRESS duration=6
[CARGOGTU] ORDER_QUEUE id=5 recipient=Ali_Celik priority=STANDARD duration=9
[CARGOGTU] ORDER_QUEUE id=6 recipient=Zeynep_Arslan priority=ECONOMY duration=15
[CARGOGTU] ORDER_QUEUE id=7 recipient=Mustafa_Koc priority=EXPRESS duration=7
[CARGOGTU] ORDER_QUEUE id=8 recipient=Elif_Demir priority=STANDARD duration=11
[CARGOGTU] ORDER_QUEUE id=9 recipient=Hasan_Ozturk priority=ECONOMY duration=18
[CARGOGTU] ORDER_QUEUE id=10 recipient=Merve_Yildiz priority=STANDARD duration=10
[COURIER-2] DELIVERY_START id=1 recipient=Ahmet_Yilmaz priority=EXPRESS
[COURIER-3] DELIVERY_START id=4 recipient=Fatma_Sahin priority=EXPRESS
[COURIER-1] DELIVERY_START id=7 recipient=Mustafa_Koc priority=EXPRESS
[COURIER-3] DELIVERY_COMPLETE id=4 recipient=Fatma_Sahin duration=3000ms
[COURIER-3] DELIVERY_START id=2 recipient=Ayse_Kaya priority=STANDARD
[COURIER-1] DELIVERY_COMPLETE id=7 recipient=Mustafa_Koc duration=3500ms
[COURIER-1] DELIVERY_START id=5 recipient=Ali_Celik priority=STANDARD
[COURIER-2] DELIVERY_COMPLETE id=1 recipient=Ahmet_Yilmaz duration=4000ms
[COURIER-2] DELIVERY_START id=8 recipient=Elif_Demir priority=STANDARD
[COURIER-1] DELIVERY_COMPLETE id=5 recipient=Ali_Celik duration=4500ms
[COURIER-1] DELIVERY_START id=10 recipient=Merve_Yildiz priority=STANDARD
[COURIER-3] DELIVERY_COMPLETE id=2 recipient=Ayse_Kaya duration=6000ms
[COURIER-3] DELIVERY_START id=3 recipient=Mehmet_Demir priority=ECONOMY
[COURIER-2] DELIVERY_COMPLETE id=8 recipient=Elif_Demir duration=5500ms
[COURIER-2] DELIVERY_START id=6 recipient=Zeynep_Arslan priority=ECONOMY
[COURIER-1] DELIVERY_COMPLETE id=10 recipient=Merve_Yildiz duration=5000ms
[COURIER-1] DELIVERY_START id=9 recipient=Hasan_Ozturk priority=ECONOMY
[COURIER-2] DELIVERY_COMPLETE id=6 recipient=Zeynep_Arslan duration=7500ms
[COURIER-2] WAITING
[COURIER-3] DELIVERY_COMPLETE id=3 recipient=Mehmet_Demir duration=10000ms
[COURIER-3] WAITING
[COURIER-1] DELIVERY_COMPLETE id=9 recipient=Hasan_Ozturk duration=9000ms
[COURIER-1] SHIFT_OVER
[COURIER-2] SHIFT_OVER
[COURIER-3] SHIFT_OVER
[CARGOGTU] SHIFT_END completed=10 cancelled=0 total_time=58000ms
[CARGOGTU] SHUTDOWN_COMPLETE
==1881==
==1881== HEAP SUMMARY:
==1881==   in use at exit: 0 bytes in 0 blocks
==1881==   total heap usage: 15 allocs, 15 frees, 18,376 bytes allocated
==1881==
==1881== All heap blocks were freed -- no leaks are possible
==1881==
==1881== For lists of detected and suppressed errors, rerun with: -s
==1881== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Figure 5: Scenario 4 — full Valgrind output showing 0 errors and no leaks

5.5 Scenario 5 — N=10, 20orders_mix.txt, repeated runs

- **Command:** `for i in 1 2 3 4 5; do ./cargoGTU -n 10 -i ... -s stats_${i}.txt; done`
- **Verification:** SHIFT_END completed=20 cancelled=0 total_time=19500ms is identical across all runs. No duplicate DELIVERY_START id= within any single run. Priority ordering correct under high concurrency.
- **Screenshot:**

```

Scenario 5 - repeats

$ for i in 1 2 3 4 5; do ./cargoGTU -n 10 -i HW5_test_files/tests/20orders_mix.txt -s captures/_stats_run${i}.txt | grep SHIFT_END; done

[CARGOGTU] SHIFT_END completed=20 cancelled=0 total_time=19500ms
[CARGOGTU] SHIFT_END completed=20 cancelled=0 total_time=19500ms
[CARGOGTU] SHIFT_END completed=20 cancelled=0 total_time=19500ms
[CARGOGTU] SHIFT_END completed=20 cancelled=0 total_time=19500ms
[CARGOGTU] SHIFT_END completed=20 cancelled=0 total_time=19500ms

$ "/mnt/c/Users/Cengiz/Desktop/Spring 2026/CSE344-Systems_Programming/HWs/HW5/cargoGTU" -n 10 -i
"/mnt/c/Users/Cengiz/Desktop/Spring
2026/CSE344-Systems_Programming/HWs/HW5/HW5_test_files/tests/20orders_mix.txt" -s captures/_stats_final.txt
[CARGOGTU] SHIFT_START couriers=10 orders=20
[CARGOGTU] ORDER_QUEUED id=1 recipient=Amet_Yilmaz priority=EXPRESS duration=1
[CARGOGTU] ORDER_QUEUED id=2 recipient=Ayse_Kaya priority=ECONOMY duration=3
[CARGOGTU] ORDER_QUEUED id=3 recipient=Mehmet_Demir priority=STANDARD duration=2
[CARGOGTU] ORDER_QUEUED id=4 recipient=Fatma_Sahin priority=EXPRESS duration=1
[CARGOGTU] ORDER_QUEUED id=5 recipient=Ali_Celik priority=STANDARD duration=3
[CARGOGTU] ORDER_QUEUED id=6 recipient=Zeynep_Arslan priority=ECONOMY duration=2
[CARGOGTU] ORDER_QUEUED id=7 recipient=Mustafa_Koc priority=EXPRESS duration=1
[CARGOGTU] ORDER_QUEUED id=8 recipient=Elif_Demir priority=STANDARD duration=2
[CARGOGTU] ORDER_QUEUED id=9 recipient=Hasan_Ozturk priority=ECONOMY duration=3
[CARGOGTU] ORDER_QUEUED id=10 recipient=Merve_Yildiz priority=EXPRESS duration=1
[CARGOGTU] ORDER_QUEUED id=11 recipient=Emre_Celik priority=STANDARD duration=2
[CARGOGTU] ORDER_QUEUED id=12 recipient=Selin_Kaya priority=ECONOMY duration=3
[CARGOGTU] ORDER_QUEUED id=13 recipient=Burak_Arslan priority=EXPRESS duration=1
[CARGOGTU] ORDER_QUEUED id=14 recipient=Dilan_Ozkan priority=STANDARD duration=2
[CARGOGTU] ORDER_QUEUED id=15 recipient=Kemal_Sahin priority=ECONOMY duration=3
[CARGOGTU] ORDER_QUEUED id=16 recipient=Rana_Yilmaz priority=EXPRESS duration=2
[CARGOGTU] ORDER_QUEUED id=17 recipient=Tarik_Demir priority=STANDARD duration=1
[CARGOGTU] ORDER_QUEUED id=18 recipient=Pinar_Koc priority=ECONOMY duration=2
[CARGOGTU] ORDER_QUEUED id=19 recipient=Oguz_Yildiz priority=EXPRESS duration=3
[CARGOGTU] ORDER_QUEUED id=20 recipient=Ceren_Arslan priority=STANDARD duration=1
[COURIER-1] DELIVERY_START id=1 recipient=Amet_Yilmaz priority=EXPRESS
[COURIER-2] DELIVERY_START id=4 recipient=Fatma_Sahin priority=EXPRESS
[COURIER-3] DELIVERY_START id=7 recipient=Mustafa_Koc priority=EXPRESS
[COURIER-4] DELIVERY_START id=10 recipient=Merve_Yildiz priority=EXPRESS
[COURIER-5] DELIVERY_START id=13 recipient=Burak_Arslan priority=EXPRESS
[COURIER-6] DELIVERY_START id=16 recipient=Rana_Yilmaz priority=EXPRESS
[COURIER-7] DELIVERY_START id=19 recipient=Oguz_Yildiz priority=EXPRESS
[COURIER-8] DELIVERY_START id=3 recipient=Mehmet_Demir priority=STANDARD
[COURIER-9] DELIVERY_START id=5 recipient=Ali_Celik priority=STANDARD
[COURIER-10] DELIVERY_START id=8 recipient=Elif_Demir priority=STANDARD
[COURIER-1] DELIVERY_COMPLETE id=1 recipient=Amet_Yilmaz duration=500ms
[COURIER-1] DELIVERY_START id=11 recipient=Emre_Celik priority=STANDARD
[COURIER-2] DELIVERY_COMPLETE id=4 recipient=Fatma_Sahin duration=500ms
[COURIER-2] DELIVERY_START id=14 recipient=Dilan_Ozkan priority=STANDARD
[COURIER-3] DELIVERY_COMPLETE id=7 recipient=Mustafa_Koc duration=500ms
[COURIER-3] DELIVERY_START id=17 recipient=Tarik_Demir priority=STANDARD
[COURIER-4] DELIVERY_COMPLETE id=10 recipient=Merve_Yildiz duration=500ms
[COURIER-4] DELIVERY_START id=20 recipient=Ceren_Arslan priority=STANDARD
[COURIER-5] DELIVERY_COMPLETE id=13 recipient=Burak_Arslan duration=500ms
[COURIER-5] DELIVERY_START id=2 recipient=Ayse_Kaya priority=ECONOMY
[COURIER-6] DELIVERY_COMPLETE id=16 recipient=Rana_Yilmaz duration=1000ms
[COURIER-6] DELIVERY_START id=6 recipient=Zeynep_Arslan priority=ECONOMY
[COURIER-3] DELIVERY_COMPLETE id=17 recipient=Tarik_Demir duration=500ms
[COURIER-3] DELIVERY_START id=9 recipient=Hasan_Ozturk priority=ECONOMY
[COURIER-4] DELIVERY_COMPLETE id=20 recipient=Ceren_Arslan duration=500ms
[COURIER-4] DELIVERY_START id=12 recipient=Selin_Kaya priority=ECONOMY
[COURIER-10] DELIVERY_COMPLETE id=8 recipient=Elif_Demir duration=1000ms
[COURIER-10] DELIVERY_START id=15 recipient=Kemal_Sahin priority=ECONOMY
[COURIER-8] DELIVERY_COMPLETE id=3 recipient=Mehmet_Demir duration=1000ms
[COURIER-8] DELIVERY_START id=18 recipient=Pinar_Koc priority=ECONOMY
[COURIER-1] DELIVERY_COMPLETE id=11 recipient=Emre_Celik duration=1000ms
[COURIER-1] WAITING
[COURIER-7] DELIVERY_COMPLETE id=19 recipient=Oguz_Yildiz duration=1500ms
[COURIER-7] WAITING
[COURIER-2] DELIVERY_COMPLETE id=14 recipient=Dilan_Ozkan duration=1000ms
[COURIER-2] WAITING
[COURIER-9] DELIVERY_COMPLETE id=5 recipient=Ali_Celik duration=1500ms
[COURIER-9] WAITING
[COURIER-5] DELIVERY_COMPLETE id=2 recipient=Ayse_Kaya duration=1500ms
[COURIER-5] WAITING
[COURIER-8] DELIVERY_COMPLETE id=18 recipient=Pinar_Koc duration=1000ms
[COURIER-8] WAITING
[COURIER-6] DELIVERY_COMPLETE id=6 recipient=Zeynep_Arslan duration=1000ms
[COURIER-6] WAITING
[COURIER-4] DELIVERY_COMPLETE id=12 recipient=Selin_Kaya duration=1500ms
[COURIER-4] WAITING
[COURIER-10] DELIVERY_COMPLETE id=15 recipient=Kemal_Sahin duration=1500ms
[COURIER-10] WAITING
[COURIER-3] DELIVERY_COMPLETE id=9 recipient=Hasan_Ozturk duration=1500ms
[COURIER-3] SHIFT_OVER
[COURIER-4] SHIFT_OVER
[COURIER-2] SHIFT_OVER
[COURIER-10] SHIFT_OVER
[COURIER-5] SHIFT_OVER
[COURIER-8] SHIFT_OVER
[COURIER-6] SHIFT_OVER
[COURIER-1] SHIFT_OVER
[COURIER-9] SHIFT_OVER
[COURIER-7] SHIFT_OVER
[CARGOGTU] SHIFT_END completed=20 cancelled=0 total_time=19500ms
[CARGOGTU] SHUTDOWN_COMPLETE

$ cat captures/_stats_final.txt
SHIFT_SUMMARY
Total orders : 20
Completed : 20
Cancelled : 0
Total time : 19500ms
Avg per order : 975ms

COURIER_STATS
Courier-1 completed=2 total_time=1500ms
Courier-2 completed=2 total_time=1500ms
Courier-3 completed=3 total_time=2500ms
Courier-4 completed=3 total_time=2500ms
Courier-5 completed=2 total_time=2000ms
Courier-6 completed=2 total_time=2000ms
Courier-7 completed=1 total_time=1500ms
Courier-8 completed=2 total_time=2000ms
Courier-9 completed=1 total_time=1500ms
Courier-10 completed=2 total_time=2500ms

```

Figure 6: Scenario 5 — five repeated runs, consistent completed=20 and full output of last run

6. Challenges & Solutions

1. Natural termination without deadlock. When the queue drains while `g_shutdown` is still false, couriers strictly blocked on “wait when empty” would deadlock. The solution is to detect completion (`completed + cancelled >= total_orders`) inside the courier’s `queue_mutex` critical section, set `g_shutdown = 1`, and `pthread_cond_broadcast` so all waiters terminate cleanly without busy-waiting.

2. Safe SIGINT with consistent pending count. A naive asynchronous handler risks `async-signal-unsafe` calls; cancelling without holding the mutex risks inconsistent `pending_orders` count. The solution uses a dedicated signal thread + `pthread_sigmask`, cancels orders only while holding `g_queue_mutex`, broadcasts after drain, and uses `pthread_kill` + `g_natural_shutdown` to exit `sigwait` cleanly on normal shutdown.

3. Log line corruption under concurrency. `printf`-based logs built from fragments interleave across threads even with stream locks. The solution wraps each logical output line in a single `g_log_mutex` critical section, matching the instructor clarification that every `printf` forming one `stdout` line must be protected together.